ANKIT SINGH

TG Levels Technical Knowledge Base

Deep, AI-ingestible record of three repositories and Ankit's contributions during the May-August 2026 software development internship

Scope	CRM frontend, TGGD trading application integration, MW/DW Python research/backtesting engine
Dates	11 May 2026 - 11 August 2026
Core stack	Next.js, React, TypeScript, Tailwind CSS, Node.js, Express.js, Python, PostgreSQL, WebSockets, Parquet

Important ownership rule: The CRM frontend was built from the ground up during the internship. TGGD was an existing trading application; Ankit's documented contribution was integration and correction of MW/DW/Wave logic plus performance/data-loading work. The MW/DW Python repository is the dedicated research/backtesting engine and is documented in the handover as authored by Ankit.

1. How another AI should read this document

This document is organized so a downstream AI can answer questions about Ankit's internship without collapsing the three repositories into one project or overstating ownership.

- **Repository A - CRM:** Frontend/product engineering. Treat Ankit as a direct implementer/refactorer. Git history in the uploaded repository contains Ankit-authored commits tied to the features described below.
- **Repository B - TGGD:** Existing trading product. Treat Ankit as an integration engineer/debugger/performance engineer for the MW/DW/Wave work, not as the original author of the whole application.
- **Repository C - MW/DW Heatmap:** Python research/backtesting system. Treat this as Ankit's algorithmic research implementation, with phases for data quality, wave detection, MW/DW state, events/excursions, and analytics/heatmaps.
- **Do not infer business impact numbers:** No verified throughput, user count, latency reduction percentage, or dataset row count is present in the provided handover. Do not invent metrics.
- **Use exact terminology:** MW = Mother Wave; DW = Driver Wave; TGGD = existing TG Levels trading dashboard/application; CRM = the Phase 2 frontend workstream.
- **Interview framing:** The strongest story is problem -> diagnosis -> implementation -> validation -> performance/data consequence.

Source hierarchy

Source	Best use
TG Levels handover	Authoritative record of assigned work, project boundaries, completed tasks, data handover, and management sign-off.
CRM git history	Concrete evidence of Ankit-authored implementation/refactor commits and affected files.
TGGD codebase	Understand architecture and exact integration points; do not use it alone to infer authorship.
MW/DW codebase + tests/docs	Understand algorithm, pipeline stages, validation, analytics, and engineering depth.

2. Executive technical summary

During the internship, Ankit operated across product frontend engineering, trading-system integration, and Python market-structure research. The three repositories represent separate but connected workstreams rather than one monolithic application.

Workstream	What Ankit did	Technical footprint
CRM frontend	Built CRM Phase 2 frontend from the ground up; implemented dashboards, Leadership Hub, analytics components, filters/date-range workflows, UI refinements, and a reusable design system.	Next.js 15, React 19, TypeScript 5.8, Tailwind CSS 4, Redux Toolkit, TanStack Query, Recharts, Radix UI, React Hook Form, Zod, Axios.
TGGD integration	Integrated validated MW/DW/Wave logic into an existing Node/Express trading application; fixed strategy edge cases, historical candle handling, chart range behavior, and recomputation/data-loading issues.	React 18, Lightweight Charts 4.1, Node.js, Express 4, Socket.IO, PostgreSQL, ParquetJS, Fyers API, backtesting package.
MW/DW research engine	Implemented a modular Python research pipeline covering data quality, Pine-compatible EMA/pivot/wave detection, Mother Wave/Driver Wave lifecycle, R/E/F events, excursion metrics, and Phase 5 analytics/heatmaps.	Python 3.12, Polars, PyArrow, NumPy, SciPy, Pydantic, DuckDB, Plotly, Matplotlib, pytest, mypy, Ruff.

Most important engineering theme

The internship demonstrates a useful combination: building UI from scratch, translating strategy/research logic across languages (Python -> JavaScript), validating algorithmic behavior, and then handling practical data-volume and rendering constraints in a real charting application.

3. Repository A - TG Levels CRM Frontend

Repository: crm-frontend-tglevels

Mission

A new CRM Phase 2 frontend intended to support dashboard analytics, leadership/sales views, lead workflows, filtering, communication-related pages, and reusable UI infrastructure. The handover explicitly describes the frontend as developed from the BRD/FRD/UI-UX handoff and implemented with Next.js, TypeScript and Tailwind CSS.

Architecture at a glance

Layer	Observed implementation
Application	Next.js app router with authenticated CRM routes under src/app/(authenticated)/crm/...
UI	Reusable UI components, Radix primitives, Tailwind utility styling, brand tokens, CRM card container, filter dropdown/chip components.
State	React Context for CRM filters; Redux Toolkit store also exists for call-related state.
Data / interaction	Axios, date-fns, deterministic seeded mock metrics in FilterContext, interactive dashboard controls.
Forms / validation	React Hook Form, Zod and @hookform/resolvers are installed and available.
Visualization	Recharts and custom chart components such as ConversionFunnel, EngagementFunnel, FailureFlow, ForecastChart, StageVelocity and IntelligencePanel.
Quality tooling	ESLint, Prettier, TypeScript typecheck, Vitest, Testing Library.

Primary CRM contributions

- **Requirements to implementation:** studied CRM Phase 2 BRD/FRD/sequence diagrams and UI/UX handoff; worked with teammates on the frontend feature list and mapped requirements to supplied designs.
- **Dashboard build:** implemented the dashboard and major sections covering forecast/metrics, conversion funnel, hourly funnel/table, failure flow, stage velocity, engagement, intelligence, and supporting reusable components.

- **Leadership Hub:** implemented leadership/sales performance views including leaderboard/ranking, funnel metrics and target/progress presentation.
- **Date filtering:** introduced date-range popups and standardized timeframe handling across dashboard components; historical-view behavior was explicitly fixed.
- **Shared filter architecture:** created/used FilterContext with timeframe, date basis, comparison, team lead, assignment and source state, plus apply/reset semantics and reusable FilterDropdown/FilterChip components.
- **UI refinement:** handled notifications, profile/logout/navigation behavior, topbar/sidebar refinements, and visual consistency across modules.
- **Design-system refactor:** commit 99e2447 standardized brand tokens and typography utilities, extended Button variants/sizes, added reusable CrmCard, FilterDropdown and FilterChip, refactored multiple subpages, and added a UI production-readiness review.

Representative CRM code pattern - filter state

```
interface FilterState {
  timeframe: { start: string; end: string };
  dateFilterLabel: string;
  dateBasis: string;
  comparison: { enabled: boolean; value: string };
  teamLead: string;
  assignment: string;
  source: string;
}

const updateFilter = (key, value) =>
  setFilters(prev => ({ ...prev, [key]: value }));
```

Representative CRM code pattern - seeded metric generation

```
const seed = `${start}-${end}-${dateBasis}-${teamLead}-${assignment}-${source}`;
const rand = seededRandom(seed);
const days = Math.max(1, diffDays(start, end));
const mul = days / 7;

// dashboard metrics and leaderboard are derived deterministically
// from the active filter state for stable UI behavior.
```

4. CRM contribution evidence and file map

The uploaded CRM repository contains the project git history. Ankit-authored commits are visible in the repository and directly support the following feature sequence.

Commit	Observed change	Why it matters
cc3b7a2	Initial CRM frontend setup	Evidence of early direct contribution to the CRM project structure and core pages.
113c8d8	Dashboard / chats / filter-bar changes	Shows active work on core CRM application modules.
ce113f0	Dashboard refining	Focused work on FailureFlow and StageVelocity.
d1624b4	UI refinements	Adjusted borders, alignment, search height, colors, chart spacing, leadership page, and filter bar.
7a2f900	CreateCampaignModal	Added broadcast center campaign creation flow.
de0f567	Give access button feature	Added permission/action UI behavior in chat/dashboard areas.
0833dd2	Date range popup + historical view fix + tooltips	Direct evidence for analytics filtering and usability improvements.
6361e11	Info tooltips	Added explanatory UI to analytics components.
8abcf00	Date range popup on conversion/failure funnels	Extended date selection behavior across funnel components.
a089e5e	Date range popup on StageVelocity/Engagement	Broadened date-range behavior across analytics modules.

Commit	Observed change	Why it matters
de16259	Card consistency / drop-off / StageVelocity / Engagement fixes	Bug fixing and visual consistency work.
807dc90	Standardize date filter UI and dashboard cards	Cross-component standardization and reusable patterns.
3c74768	Filter bar enhancements	Active chips, single-click removal, toggle integration across funnel components.
9060a31	Notification feature	Direct implementation of notification UI feature.
99e2447	Design-system standardization	831 additions / 234 deletions across 18 files; reusable components and production-readiness audit.

Key source paths for a downstream AI

- src/app/(authenticated)/crm/dashboard/components/ConversionFunnel.tsx
- src/app/(authenticated)/crm/dashboard/components/EngagementFunnel.tsx
- src/app/(authenticated)/crm/dashboard/components/FailureFlow.tsx
- src/app/(authenticated)/crm/dashboard/components/ForecastChart.tsx
- src/app/(authenticated)/crm/dashboard/components/HourlyTable.tsx
- src/app/(authenticated)/crm/dashboard/components/IntelligencePanel.tsx
- src/app/(authenticated)/crm/dashboard/components/StageVelocity.tsx
- src/app/(authenticated)/crm/leadership/page.tsx
- src/modules/crm/components/FilterBar.tsx
- src/modules/crm/context/FilterContext.tsx
- src/components/ui/DateRangePopup.tsx
- src/modules/crm/components/filters/FilterDropdown.tsx
- src/modules/crm/components/filters/FilterChip.tsx
- src/styles/tokens.css

What this proves

Ankit was not limited to styling isolated screens. The repository shows component-level feature work, cross-component state/filter behavior, reusable UI infrastructure, and a large design-system refactor. The handover independently confirms CRM dashboard, Leadership Hub, standardized filtering, and UI interaction refinements.

5. Repository B - TGGD Trading Application

Boundary: This was an existing TG Levels trading application. Ankit did not build the entire TGGD system. His contribution was to bring validated MW/DW/Wave logic from the research work into this existing application and solve the integration/data-loading/performance issues exposed by that work.

Observed architecture

Area	Implementation found in repo
Frontend	React 18, React Router, Lightweight Charts, Socket.IO client. ChartsPage, CandleChart, WavesIndicator, MotherWave navigation, wave stats, signal tables, strategy/backtest/report pages.
Backend	Node.js, Express, Axios, Socket.IO, Fyers API v3, PostgreSQL client, ParquetJS, XLSX, rate limiting.
Database	PostgreSQL/TimescaleDB-oriented schema with candle storage, validation, recovery, repair log, derivatives storage and migrations.
Backtesting	Separate backtest package with dataLoader, Parquet loader, indicators, strategy, engine and report layers.
Market data	Fyers client/tick stream, historical candles, derivative gap filling, expiry lifecycle and archive/export paths.

Relevant routes/services

- backend/src/routes/chartRouter.js - chart range logic, Mother Wave-aware extension, payload construction.
- backend/src/services/motherwave.js - wave computation, MW/DW state machine, succession rules and Fibonacci/invalidation logic.
- backend/src/services/backtestRunner.js and backend/src/routes/backtestRouter.js - backtest execution/reporting integration.
- backend/src/fyers/tickStream.js - live market-data subscription path.
- frontend/src/components/CandleChart.js - Lightweight Charts rendering.
- frontend/src/indicators/WavesIndicator.js - Pine-compatible wave calculation/overlay logic.
- frontend/src/pages/ChartsPage.js - chart refresh/extension behavior and indicator coordination.
- frontend/src/hooks/useSocket.js - chart fetches, reconnect behavior, Mother Wave-aware requests.
- database/src/candleValidation.js, validationEngine.js, recoveryEngine.js - data integrity and repair infrastructure.

6. TGGD contribution - MW/DW/Wave integration and chart performance

The handover states that the verified MW/DW logic was integrated into the existing TG Levels trading dashboard using Node.js/Express and WebSocket support. It also records correction of the previously integrated Mother Wave logic, addition of Driver Wave logic, historical candle improvements, CSV-to-Parquet work, and dataset validation.

Contribution chain

- **1. Translation/integration:** the Python research logic became an application-side implementation in JavaScript/Node, while the frontend overlay also needed compatible wave calculations.
- **2. Rule correctness:** MW and DW state had to respect wave sequence, successor conditions, invalidations, direction/size relationships, and qualifying cuts.
- **3. Historical context:** calculations for a current MW can require candles older than the standard chart display window; the system therefore needed a controlled history extension strategy.
- **4. Chart performance:** the initial fixed three-month chart window caused trouble when an active MW originated earlier. A naive extension path led to larger candle arrays and recomputation, stressing Lightweight Charts on lower timeframes.
- **5. Better range selection:** chartRouter computes an effective start as the minimum of the normal display start, MW start, and explicit target start, but only extends when that value is older than the normal window.

- **6. Lower-timeframe guard:** backward extension is limited to 15m and 60m resolutions; 1m/3m/5m/1D/1W do not inherit the same extension behavior, protecting chart responsiveness.

Representative code logic - effective chart start

```
normalWindowStart = now - CHART_DB_WINDOW_DAYS * 24h
                    # default environment value is 90 days

candidates = [normalWindowStart]
if finite(mwStart): candidates.push(mwStart)
if finite(extendTo): candidates.push(extendTo)

floored = min(candidates)
if floored < normalWindowStart:
    effectiveExtendTo = floored

# Important: a younger MW/DW target does NOT shrink the normal 3-month window.
```

Representative frontend rule

```
const MW_RANGE_RESOLUTIONS = new Set([15, 60]);

refresh(symbol, resolution, {
  extendTo: urlWaveTarget?.fromMs,
  includeMotherWave: MW_RANGE_RESOLUTIONS.has(Number(resolution)),
});
```

Important distinction

Do not describe this as simply "load from the MW start". The actual policy preserves the normal three-month baseline whenever the MW is younger than three months. An older MW extends the range back to its start. This is the key design decision that makes the feature useful without creating unnecessary data volume.

7. TGGD MW/DW/Wave rule implementation - technical model

backend/src/services/motherwave.js contains a single-source-of-truth style implementation for Mother Wave / Driver Wave detection. The file comments explicitly state that the wave detector mirrors the Python repository conventions and that MW/DW state is processed sequentially over full wave history.

Wave configuration mirrored across Python and JavaScript

Parameter	Observed value / role
minimumWaveBars	3; minimum span before another pivot can be confirmed.
useFivePointValidation	false in the mirrored chart config; determines 1-point vs 5-point neighborhood validation.
equalHighLowTolerancePct	0.2%; equal high/low classification tolerance.
firstPivotSearchLimit	300 bars; initial pivot search cap.
laterPivotSearchLimit	480 bars; later pivot search cap.
anyColorBearTrigger	true; body-mid-below-EMA condition can trigger bearish behavior independent of candle color.
confirmBar	false; current configuration does not require the previous bearish state for confirmation.

Mother Wave succession rules

- **Initialization:** MW starts after the first 50 completed waves under the configured initial-wave count.
- **S1:** if a candidate wave is larger than the current MW, it can immediately replace the MW.
- **S2:** same-direction candidate of at least 0.5x MW size plus a qualifying cut beyond the current MW -0.618 Fibonacci level.
- **S3:** opposite-direction candidate of at least 0.5x MW size plus a qualifying cut beyond the current MW 1.234 level.
- **DW candidate:** a temporally eligible wave with size at least 0.3x the current MW size, subject to forward-only lifecycle rules.
- **DW invalidation:** a qualifying cut beyond the DW's own 1.234 level can deactivate the DW.

Qualifying cut logic

```
body = abs(close - open)
if body < qualifying_cut_body_atr_ratio * previous_bar_ATR:
    reject("BODY_TOO_SMALL")

if close not beyond target level:
    reject("CLOSE_NOT_BEYOND")

if daily and gap across target level qualifies:
    record cut evidence

# This prevents a nominal level touch from being treated as a qualifying cut.
```

8. Repository C - MW/DW Heatmap Research Engine

Repository: w_mw_dw_heatmap-main

Purpose

A modular-monolith Python research system that turns validated market candles into deterministic wave/MW/DW/event/analytics artifacts. The README explicitly documents a staged roadmap: Phase 1 data quality, Phase 2 Pine-compatible wave detection, Phase 3 MW/DW, Phase 4 events/excursions, and Phase 5 analytics/heatmaps/dashboard.

Architecture

Package	Responsibility
domain	Candle contracts/validation, indicators, waves, Mother Wave, Driver Wave, events, excursions, analytics. Domain code does not perform file I/O.
application	Coordinates services and pipelines; owns orchestration, artifact generation, and integrity gates.
infrastructure	CSV/manifest/logging adapters and external I/O concerns.
config	Typed configuration models and YAML loading.
scripts	Convenience runners for phases 1-5, validation, dataset discovery, dashboard and personal export.
tests	Unit, integration, and regression coverage for wave, MW/DW, event, validation, and Phase 5 dashboard contracts.

Dependencies

Python >= 3.12; Polars, PyArrow, NumPy, SciPy, Pydantic, PyYAML, Typer, Rich, structlog, dateutil, Matplotlib, Plotly, Kaleido, DuckDB; dev stack includes pytest, coverage, Hypothesis, Ruff and mypy.

Phased pipeline

- **Phase 1:** audit/clean candles, validate schema/parse errors, duplicates, ordering, session completeness, timestamp semantics, quarantine invalid rows, and preserve raw inputs as immutable source-of-truth.
- **Phase 2:** Pine-compatible EMA9/pivot/completed-wave detection; wave classification and confirmation delays.
- **Phase 3:** sequential Mother Wave / Driver Wave periods and state transitions using ATR/fib/cut/eligibility rules.
- **Phase 4:** Q/T/level assignment, independent R/E/F event scanning, excursion exits/censoring, event metrics and audit reports.
- **Phase 5:** aggregate event statistics, qualification, key-area decisions, heatmap cells, dashboard JSON and offline HTML dashboard.

9. Python wave detector - implementation details

The core wave detector lives in domain/waves/detector.py. It is sequential, deterministic, and designed to mirror Pine behavior rather than using a generic peak-finding library.

- **EMA basis:** computes Pine-compatible EMA values over candle highs and lows.
- **Leg state:** alternates between searching for a pivot LOW to start an up-leg and a pivot HIGH to close an up-leg.
- **Bull trigger:** close > open and close > EMA-high.
- **Bear trigger:** body midpoint below EMA-low, with optional confirmation behavior controlled by configuration.
- **Actual extreme:** searches backward inside a capped span to find the actual high/low bar, not merely the trigger candle.
- **Neighborhood validation:** candidate pivot must survive 1-point or 5-point validation depending on configuration.
- **Classification:** previous comparable high/low is used to classify HH/LH/EH or HL/LL/EL.
- **Completed wave record:** stores origin/tip pivots, prices, timestamps, size, actual span, confirmation bar, and confirmation delay.
- **No look-ahead objective:** regression tests explicitly include test_wave_detector_no_lookahead.py.

Representative wave-detection excerpt

```
green_bull = candle.close > candle.open and candle.close > ema_high[current_index]
bear_now, bear_trigger = _bear_signals(candle, ema_low[current_index], config, previous_bear_now)

if leg_state == -1 and green_bull and wave_long_enough:
    kind = PivotKind.LOW
elif leg_state == 1 and bear_trigger and wave_long_enough:
    kind = PivotKind.HIGH

actual_index, price, best_offset = _actual_extreme(...)
```

```
if _candidate_valid(...):
    classification = classify_pivot(...)
    ...
    waves.append(_completed_wave(...))
```

10. Python MW/DW engine - state model

The Python Mother Wave engine consumes the chronological completed-wave list and walks candle-by-candle to maintain MW and DW lifecycle state. The domain layer includes fib pricing, qualifying-cut evaluation, succession, driver qualification, invalidation, and period/transition records.

Key state semantics

- **ATR-aware cuts:** qualifying cut evidence is evaluated against the previous-bar ATR to avoid treating tiny bodies as decisive breaks.
- **Fib levels:** Mother Wave state uses ratios such as -0.618 and 1.234 for succession/invalidation checks; the same geometry is used downstream for event/grid analysis.
- **Succession:** S1/S2/S3 are mutually meaningful replacement mechanisms; the successor wave becomes the new MW and cannot also become its own DW.
- **Forward-only DW:** Driver Wave eligibility is temporal and size-constrained; historical waves do not retroactively become active drivers for a current MW.
- **Lifecycle periods:** the engine records activation/deactivation/replacement/end reasons so downstream analysis can reconstruct when each state was active.
- **Configuration-driven:** ratios and thresholds are externalized via MWDW configuration rather than hardcoding all strategy constants.

Representative mathematical helper

```
def fib_price(wave, ratio):
    return wave.tip_price - ratio * (wave.tip_price - wave.origin_price)
```

Important tests

- tests/unit/test_mother_driver_rules.py
- tests/unit/test_forward_only_mwdw.py
- tests/unit/test_qualifying_cut.py
- tests/integration/test_phase3_mwdw_pipeline.py
- tests/regression/test_phase3_mwdw_targets.py
- tests/regression/test_phase3_replay_and_mirror.py

11. Phase 4 events, excursions and Phase 5 analytics

After waves and MW/DW state, the engine builds a structured event layer. Event types are R, E, and F; the analytics engine works over timeframes 15min, 60min and daily, quadrants Q1-Q4, T1-T7 zones, and level grids.

Event model

- **Q/T/level assignment:** events are placed into grid cells based on quadrant, T-zone and Fibonacci/extension level.
- **R/E/F events:** each family has its own level set and downstream aggregation rules.
- **Excursion metrics:** D, MFE, MAE and A_with/A_against style measures support comparison of movement with or against MW.
- **Eligibility gates:** incomplete rows, outside-grid rows, off-level rows, and right-censored data are filtered before Phase 5 aggregation.

- **Qualification:** uses timeframe profiles, sample counts, Wilson lower bounds, asymmetry thresholds, recent-adjusted success, cross-timeframe checks and optional shuffle criteria.
- **Heatmap output:** produces per-timeframe cells, key-area claims, consolidated decisions, milestones, dashboard JSON and offline HTML dashboards.

Observed analytics dimensions

Dimension	Values / role
Timeframes	15min, 60min, daily
Event types	R, E, F
Quadrants	Q1, Q2, Q3, Q4
T zones	T1 through T7
R/E levels	-0.236, 0, 0.236, 0.382, 0.5, 0.618, 0.786, 1.0
F levels	F_NEG0236, F_1234

Key tests

- tests/unit/test_phase4_events.py
- tests/unit/test_phase5_analytics.py
- tests/integration/test_phase5_output_contract.py
- tests/integration/test_phase5_key_area_dashboard.py
- tests/integration/test_phase5_legacy_ui_lock.py

12. Data engineering and validation

Data handling is a major part of the internship story. The handover records conversion of historical equity and NIFTY options CSV datasets to partitioned Parquet, plus row-count/date-range/checksum auditing and gap identification. The TGGD repository also contains a database validation/recovery subsystem and Parquet readers/writers.

Python research data-quality rules

- Raw inputs are immutable and treated as source-of-truth.
- Validated data, quarantine records, manifests, reports and phase outputs are separated from raw inputs.
- Invalid OHLC rows and objectively incomplete trailing regular sessions are excluded in strict mode.
- Special/irregular sessions are preserved and labeled unless a study explicitly excludes them.
- NIFTY50 index volume is unavailable/zero and is never fabricated.
- Daily 05:30 IST timestamps are preserved as source UTC-midnight epochs while trading date is derived separately.
- OHLC differences between daily/intraday views are warned about rather than auto-repaired.

TGGD database/data infrastructure

- database/src/candleValidation.js - candle-level validation.
- database/src/validationEngine.js - broader validation checks.
- database/src/recoveryEngine.js and repairLog.js - recovery/repair mechanics.
- database/src/candleStore.js and dataRouter.js - candle persistence/access.
- database/src/derivativesStore.js, backfillDerivatives.js and expiryLifecycle.js - derivatives lifecycle and gap filling.
- backend/src/archive/parquetExport.js - archival/export path used by the broader trading system.
- backtest/src/parquetDataLoader.js - reads Parquet option/futures data and reconstructs ATM context for backtests.

13. Validation, testing and verification mindset

The strongest evidence of engineering maturity in these repositories is not the number of technologies; it is the amount of validation around algorithmic behavior and data contracts.

- **CRM:** typecheck/lint/test scripts exist; the design-system refactor included a production-readiness audit document.
- **TGGD:** database validation, repair/recovery, derivatives integration tests, symbol parser tests, and dedicated Mother Wave tests exist in the repository.
- **MW/DW:** unit, integration, and regression tests cover pivots, completed waves, no-lookahead, MW/DW targets, forward-only behavior, events, validation, analytics, dashboard output contracts, and known NIFTY50 anomalies.
- **Cross-source verification:** handover says strategy logic was cross-checked collaboratively and validated against Pine/TradingView behavior before final implementation.
- **Regression mentality:** the MW/DW project includes replay/mirror tests and regression counts/targets so refactors can be checked against known behavior.

Example interview explanation

Question: "How did you know the MW/DW implementation was correct?"
Answer frame: "I did not rely only on visual output. The Python engine had unit/integration/regression tests for wave and MW/DW rules, and we cross-checked strategy behavior against Pine/TradingView references. I then integrated the validated behavior into the Node/Express trading application and fixed discrepancies such as existing MW behavior and DW integration."

14. End-to-end technical story connecting the three repositories

This is the best single mental model for another AI:

```
CRYPTO? NO. The actual internship flow was:

Business / strategy handover
        |
        v
CRM requirements -> BRD/FRD/UI-UX -> Next.js/TypeScript frontend

Trading strategy handover
        |
        v
Python research/backtesting engine
  Phase 1 data quality
  -> Phase 2 waves
  -> Phase 3 MW/DW
  -> Phase 4 R/E/F + excursions
  -> Phase 5 analytics/heatmaps
        |
        v
Validated MW/DW/Wave behavior
        |
        v
Existing TGGD Node/Express application
  -> chart integration
  -> real-time/WebSocket path
  -> historical data/range management
  -> performance constraints
```

What makes this technically interesting

- It crosses languages: Python research logic and JavaScript application logic must agree on wave semantics.
- It crosses layers: domain logic becomes backend services and then chart overlays/UI state.
- It exposes real-world constraints: old historical state versus fixed display windows, data volume versus rendering capacity, and correctness versus latency.

- It combines deterministic research with practical application engineering: reproducible backtests, audit artifacts, and interactive real-time charts.

15. What Ankit can credibly claim - and what he should not claim

High-confidence claims

- Built CRM Phase 2 frontend from the ground up with Next.js, TypeScript and Tailwind CSS.
- Implemented dashboard analytics and Leadership Hub UI, filters, historical date views, notifications and reusable UI infrastructure.
- Built the Python MW/DW research/backtesting pipeline and its phased data/analytics outputs.
- Implemented/validated Mother Wave, Driver Wave and Wave behaviors under the documented strategy conventions.
- Integrated MW/DW/Wave logic into the existing Node.js/Express TGGD application.
- Solved chart historical-range and performance problems, including dynamic range selection and lower-timeframe restrictions.
- Converted historical CSV data to partitioned Parquet and performed dataset audits/validation.

Claims that should be avoided without extra evidence

- "Built the entire TGGD trading platform" - false; TGGD was already built.
- "Architected the company-wide trading system" - not supported by the evidence.
- "Improved chart performance by X%" - no verified metric was provided.
- "Processed N million rows / N GB" - no verified dataset size was provided.
- "Production-grade at scale for X users" - user/throughput evidence is absent.
- "Developed a profitable trading strategy" - the repository shows research/backtesting logic, not verified profitability.

Best one-line positioning

Full-stack developer with hands-on experience building CRM software from scratch, implementing Python market-structure research/backtesting, and integrating validated trading logic into an existing Node.js/Express real-time application.

16. Resume and interview mapping

Resume area	Evidence from the three repos	Interview angle
Full Stack / CRM	Next.js, React, TypeScript, Tailwind, dashboard/leadership modules, filters, shared UI components.	Component architecture, state management, requirements-to-code, UI consistency.
Backend / Node	Express routes/services, Socket.IO, chart API, Mother Wave service.	API boundaries, real-time updates, integrating domain logic.
Python / algorithms	Wave detector, MW/DW engine, Phase 4 events, Phase 5 analytics.	Sequential state machine, deterministic processing, validation, testing.
Performance	Dynamic chart range, extension guard, lower-timeframe restriction.	How you diagnosed unnecessary computation and rendering pressure.
Data engineering	CSV -> Parquet, validation, audits, TGGD DB/recovery infrastructure.	Why Parquet, data quality, source-of-truth and auditability.
Testing	Unit/integration/regression tests in MW/DW and TGGD DB.	How correctness was established and protected during changes.

Most likely technical interview questions

- How did you implement date-range filtering in the CRM and prevent duplicated filter logic?
- What is the difference between the Python MW/DW engine and the TGGD integration?
- Why did chart extension require data older than the standard three-month window?
- Why does min(normalWindowStart, mwStart, explicitTarget) preserve the three-month baseline?
- Why were lower timeframes restricted for backward extension?
- How did you validate that the JavaScript wave implementation matched the Python/Pine behavior?
- What is a qualifying cut and why use the prior-bar ATR?
- How does a Driver Wave become eligible and how can it be invalidated?
- Why convert historical CSV data to Parquet?
- How did you audit the historical datasets before using them in the strategy research?

17. Repository inventory for future AI retrieval

The uploaded archives contain the following approximate file counts excluding .git metadata, node_modules, and Python bytecode caches: CRM 151 files, TGGD 152 files, MW/DW 144 files. The lists below are the key retrieval anchors rather than every utility file.

Repo	Key retrieval anchors
CRM	src/app/(authenticated)/crm/dashboard/components/*, src/app/(authenticated)/crm/leadership/page.tsx, src/modules/crm/components/FilterBar.tsx, src/modules/crm/context/FilterContext.tsx, src/modules/crm/components/filters/*, src/components/ui/DateRangePopup.tsx, src/styles/tokens.css, package.json, .git history.
TGGD	backend/src/routes/chartRouter.js, backend/src/services/motherwave.js, backend/src/server.js, backend/src/fyers/tickStream.js, backend/src/services/backtestRunner.js, frontend/src/pages/ChartsPage.js, frontend/src/components/CandleChart.js, frontend/src/indicators/WavesIndicator.js, frontend/src/hooks/useSocket.js, database/src/*, backtest/src/*
MW/DW	README.md, docs/architecture.md, docs/data_quality_rules.md, docs/phase2_wave_detection.md, docs/phase3_mw_dw_engine.md, docs/phase4_event_engine.md, docs/phase5_analytics_engine.md, src/mwdw_heatmap/domain/*, application/pipelines/*, tests/unit/*, tests/integration/*, tests/regression/*, configs/*.yaml

Suggested AI retrieval order

- Start with this document's section 15 (ownership/claims) before answering questions about what Ankit built.
- For CRM questions, inspect section 3 and then the specific file path/commit in section 4.
- For TGGD questions, inspect sections 5-7 and treat chartRouter + motherwave.js + ChartsPage + useSocket as the main integration chain.
- For Python MW/DW questions, inspect sections 8-12, then the specific domain/pipeline/test file.
- For resume wording, use section 16 to map technical evidence to a concise claim.

18. Cross-check against the official internship handover

The official employee handover identifies the designation as Software Developer Intern (MERN Stack / Full Stack / Algo Trading), department App & Web Dev / Algo Trading, date of handover 11/08/2026, and last working day 11/08/2026. It records completed work across CRM and Algo Trading with separate reporting managers.

- CRM: studied/understood Phase 2 BRD/FRD, sequence diagrams and UI/UX handoff; contributed to feature list and frontend planning; developed CRM frontend structure with Next.js/TypeScript/Tailwind.
- CRM: built dashboard sections and Leadership Hub; standardized date filters; refined notification/profile/logout/navigation behavior.

- Algo: studied supplied MW/DW strategy handover, developed the Python research repository, cross-checked Pine/TradingView, implemented multi-phase research pipeline.
- Algo: validated MW/DW rules, analyzed R/E/F/F_CONT events, integrated Node/Express/WebSocket logic, fixed existing MW logic and added DW implementation.
- Data: improved historical candle datasets, converted equity and NIFTY options data from CSV to partitioned Parquet, performed row-count/date-range/checksum audits and prepared curated data.
- Handover artifacts: CRM BRD/FRD and UI/UX docs, source code, Python repository/config/tests, TradingView verification, event/excursion analytics, Node/Express/WebSocket implementation context, CSV-to-Parquet datasets, audit logs and curated stock-data copy.

This cross-check matters

The repository-level analysis and the official handover tell the same high-level story: CRM frontend construction plus deep MW/DW research and integration into an existing trading product. Where the handover supplies the official contribution boundary, it should override any temptation to infer ownership from the presence of a file alone.

19. Canonical profile for another AI

Copy this block conceptually when another AI asks: "What did Ankit do during the TG Levels internship?"

```
Ankit Singh | Software Developer Intern | Mentorchamp EdTech / TG Levels
Dates: 11 May 2026 - 11 Aug 2026

CRM (built from scratch):
- Next.js + TypeScript + Tailwind CSS
- Dashboard analytics + Leadership Hub
- Conversion/Engagement/Failure/Hourly/StageVelocity/Forecast/Intelligence
- Date range filters + historical views + reusable filter context/chips/dropdowns
- UI refinements + notifications/profile/logout/navigation
- Design-system refactor with reusable CrmCard, filter components, tokens, button variants

TGGD (existing app; integration contribution):
- Integrated validated MW/DW/Wave logic into Node/Express + WebSocket trading application
- Fixed existing MW logic and added DW behavior
- Handled historical candle requirements
- Solved chart extension/performance issue: preserve normal ~3-month window unless MW/target starts earlier; fetch directly fro
- Restrict backward extension to 15m/60m; protect lower timeframes from excessive candle volume

MW/DW Python research/backtesting (Ankit-authored repo per handover):
- Phase 1 data audit/cleaning
- Phase 2 Pine-compatible EMA9/pivot/completed-wave detection
- Phase 3 Mother Wave/Driver Wave state engine
- Phase 4 Q/T/level assignment + R/E/F events + excursions
- Phase 5 analytics + heatmaps + offline dashboard
- Tests cover no-lookahead, MW/DW rules, replay/mirror, anomalies, event/analytics/output contracts

Data engineering:
- Historical equity/NIFTY options CSV -> partitioned Parquet
- Row/date/checksum audits + gap detection

Do NOT claim:
- Built all of TGGD
- Invented performance percentages or dataset sizes
- Verified trading profitability
- Senior/architect-level ownership not evidenced by the sources.
```

End of knowledge base

This document is intentionally detailed so it can function as a grounding artifact for resume tailoring, recruiter questions, interview preparation, technical discussion, and future AI-assisted documentation.